

SAGE — Autonomous Campus Tour Robot

2026 Senior Design Project · Valparaiso University College of Engineering

[Video Demo](#)

SAGE is a custom-built autonomous mobile robot designed to tour visitors through the first floor of Gellersen Engineering at Valparaiso University. It navigates autonomously using a 2D LIDAR map, responds to natural voice commands via an LLM pipeline, streams live video to operators, and displays an animated face on an iPad in kiosk mode. The system runs on a **Jetson Orin Nano** (JetPack 6.1) with **ROS 2 Humble** as the middleware, and a **STM32G0B1** microcontroller handling low-level motor control and sensor I/O.

Table of Contents

1. [CRITICAL: WiFi & Network Access](#)
 2. [Project Overview](#)
 3. [Repository Structure](#)
 4. [Hardware Overview](#)
 5. [System Architecture](#)
 6. [STM32 Firmware & Serial Protocol](#)
 7. [ROS 2 Subsystems](#)
 8. [Navigation: Maps & Waypoints](#)
 9. [Voice Interaction System](#)
 10. [Web Interfaces & Network Services](#)
 11. [Startup Procedures](#)
 12. [API Keys & Secrets](#)
 13. [Libraries Installation & Modifications](#)
 14. [Known Issues & Debugging Guide](#)
 15. [Porting to Another Robot](#)
 16. [Useful Resources](#)
 17. [Acknowledgments](#)
-

1. CRITICAL: WiFi & Network Access

Read this first. If SAGE loses network connectivity, everything that depends on the network (the iPad face, teleoperation, voice recording web page, knowledge base) stops working. This is the most likely maintenance event.

Why This Happens

SAGE connects to campus WiFi using a personal credential. The university IT network may change SAGE's IP address (DHCP), or WiFi may drop after a reboot. The university's IT department is also planning infrastructure changes that may require reconfiguring the WiFi connection manually.

How to Find SAGE's Current IP (When It's Connected)

Ask SAGE verbally: **"Hey Jarvis, what is your IP address?"**

SAGE will respond with its current IP(s). You can then open its interfaces from any browser on the same network.

How to Reconnect SAGE to WiFi (Headless Jetson)

By default, the Jetson boots in **headless mode** (Gnome GUI is disabled to save memory). To reconnect to WiFi from the terminal:

Option A — SSH in (if partially connected or on a wired network):

```
nmcli device wifi list
```

```
nmcli device wifi connect "SSID_NAME" password "PASSWORD"
```

Option B — Physical monitor (if SSH is not possible):

1. Power off SAGE.
2. Remove the Jetson from the robot (requires opening the lid).
3. Connect the Jetson to a monitor via HDMI, and connect a keyboard.
4. Power it on. It will boot headless (terminal only).
5. Use the nmcli commands listed in option A above.
6. Reinstall the Jetson in the robot
7. Reboot SAGE. (Flip switch at the bottom left corner of robot)
8. Teleop SAGE to docking station. Make sure she faces the wall/charger.
9. Reboot one more time. She should be ready to go after ~5 mins.

If you need to enable the Gnome (default GUI for jetson) then do the following

1. Re-enable the Gnome desktop temporarily:

```
sudo systemctl set-default graphical.target
```

```
sudo reboot
```

2. After reboot, use the Gnome network manager GUI to reconnect to WiFi.
3. Disable Gnome again to restore normal headless operation:

```
sudo systemctl set-default multi-user.target
```

```
sudo reboot
```

Remember to disable the Gnome desktop after you have activated it. It consumes a lot of memory.

After Reconnecting

Restart all SAGE services by powering down and restarting the robot at its docking station (there is a switch on the bottom left side).

Restart must be done at docking station with SAGE facing the wall/charger because that's SAGE's initial pose is set to be her docking station facing the wall.

2. Project Overview

Mission

SAGE autonomously tours visitors through the first floor of Gellersen Engineering, guides them to named locations (labs, maker spaces, cafeteria, etc.), answers questions about Valparaiso University via a local knowledge base, and supports teleoperation by an operator over a browser interface.

Key Capabilities

Capability	How It Works
Autonomous navigation	SLAM-built map + AMCL localization + Nav2 path planning

Capability	How It Works
Voice interaction	Wake word → Whisper STT → OpenAI LLM → Piper TTS
Live video streaming	USB camera → ROS 2 → web_video_server → browser
Teleoperation	Browser joystick → WebSocket → ROS 2 /cmd_vel → STM32
Animated face	React app on port 8002, iPad in kiosk mode
Eye tracking	XVF3800 mic DOA → WebSocket → face UI
Knowledge base	Django + PostgreSQL + pgvector semantic search
Battery monitoring	Serial bridge → ROS 2 topic → watchdog → email alert

Tech Stack

- **Compute:** NVIDIA Jetson Orin Nano, JetPack 6.1
- **MCU:** STM32G0B1 (ARM Cortex-M0+ @ 64 MHz)
- **Middleware:** ROS 2 Humble
- **Navigation:** Nav2, SLAM Toolbox, robot_localization (EKF)
- **AI/ML:** OpenAI API (GPT, streaming), Whisper (faster-whisper), Piper TTS, openwakeword
- **Web:** FastAPI, WebSockets, React 19 + Vite, Vanilla JS
- **Database:** PostgreSQL + pgvector, Django ORM
- **Containerization:** Docker Compose (knowledge base only)
- **Python:** 3.10.12, virtual environment at ~/ .venv

3. Repository Structure

The repository has three active branches:

Branch	Contents
jetson	All robot software: ROS 2 nodes, speech, navigation, web interfaces, startup scripts

Branch	Contents
MCU	STM32G0B1 firmware (CubeIDE project)
main	Same as Jetson. Used to have Early-stage Three.js simulation demo (historical, not used on the robot)

All robot operation described in this document refers to the **jetson branch**.

SAGE_ROBOT/ (jetson branch)

```

├── battery/
|   └── battery_watchdog.py    # Monitors /battery_state, emails on low voltage
├── config/
|   ├── nav2_params.yaml      # AMCL + Nav2 configuration (primary)
|   ├── nav2_params_mpii.yaml # Alternate Nav2 config
|   └── ekf.yaml              # Extended Kalman Filter parameters
├── description/
|   ├── sage.urdf.xacro       # Full URDF with IMU
|   └── sagewithoutimu.urdf.xacro # URDF without IMU (currently used)
├── interface/
|   ├── teleop_interface/
|   |   └── index.html         # Operator teleoperation UI (port 8001)
|   └── status_interface/     # Animated face / status UI (port 8002, React)
├── knowledge_base/
|   └── docker-compose.yml     # PostgreSQL + Django containers

```

- | └─ Dockerfile
- | └─ SAGE_KB/ # Django app: ingest, embed, search endpoints
- | └─ maps/
 - | └─ new_save_map.pgm/.yaml # Current active map (Gellersen 1st floor)
 - | └─ Good_Gelly_Save_MAP.* # Previous map backup
 - | └─ new_waypoints.yaml # Named locations (poses for Nav2)
 - | └─ waypoints.yaml # Older waypoint file
- | └─ ros2_ws/
 - | └─ src/web_teleop_bridge/
 - | └─ control_bridge.py # WebSocket ↔ ROS 2 bridge (port 8765)
 - | └─ serial_bridge.py # STM32 ↔ ROS 2 bridge (with IMU)
 - | └─ serial_bridge_without_imu.py # STM32 ↔ ROS 2 bridge (currently used)
- | └─ speech/
 - | └─ main.py # Voice system entry point
 - | └─ config.py # All configurable constants
 - | └─ system_prompt.py # LLM persona and instructions
 - | └─ waypoints.py # Waypoint definitions + descriptions (for LLM)
 - | └─ navigation.py # Nav2 async wrapper
 - | └─ tools.py # LLM tool definitions (set_goal, search, etc.)
 - | └─ streaming.py # OpenAI streaming response handler
 - | └─ events.py # Event dispatcher (arrival announcements)
 - | └─ piper_tts.py # Local TTS engine (Piper ONNX)

- | | — web_ptt.py # Push-to-talk web endpoint (port 8005)
- | | — doa_server.py # Direction-of-arrival broadcaster (port 8766)
- | | — ui_state_client.py # Publishes UI state to ROS 2 topic
- | | — logger.py # Structured logging
- | | — utils.py # Shared utilities
- | | — replace_in_stt_library/
- | | | — audio_recorder.py # Patched realtimestt file (see Section 13)
- | | | — readme.md
- | | — assets/
- | | — models/wakeword/ # Wakeword ONNX models
- | | — models/piper/ # Piper TTS ONNX models
- | — start_robot.sh # Full bring-up (nav + speech + KB + face)
- | — start_robot_with_imu.sh # Full bring-up with IMU enabled
- | — slam_robot.sh # SLAM mode (no speech/KB)
- | — slam_robot_with_imu.sh # SLAM mode with IMU
- | — python_3_10_12_requirements.txt

4. Hardware Overview

Note: Detailed mechanical and electrical specifications — chassis dimensions, motor driver circuit, wiring diagrams, 3D-printed parts, bill of materials — are covered in a separate hardware document maintained by the ME/EE team. This section gives a software-relevant summary only.

Compute & MCU

Component	Details
Jetson Orin Nano	Primary compute, runs ROS 2 and all Python services, JetPack 6.1
STM32G0B1	Motor control, dead-wheel odometry, IMU readout, battery ADC, USB CDC serial to Jetson

Sensors

Sensor	Interface	ROS 2 Topic	Rate
RPLIDAR A1M8	USB (sllidar_ros2 driver)	<code>/scan</code>	~10 Hz
Dead-wheel encoders (×2)	STM32 timers TIM1, TIM2	<code>/odom</code> (via serial bridge)	50 Hz
BNO08x IMU	I2C3 on STM32	<code>/imu/data</code> (via serial bridge)	100 Hz
USB Camera	V4L2	<code>/image_raw</code>	~30 Hz
Battery ADC	STM32 ADC1, PA4	<code>/battery_state</code> (via serial bridge)	50 Hz

Actuators

Component	Details
Differential drive motors (×2)	Controlled by ESC-style 50 Hz PWM from STM32 (TIM14/TIM15)
Speaker	Integrated into robot body, driven via ALSA/PulseAudio on Jetson

Microphone

The robot uses an **XMOS XVF3800 USB microphone array**. This mic provides:

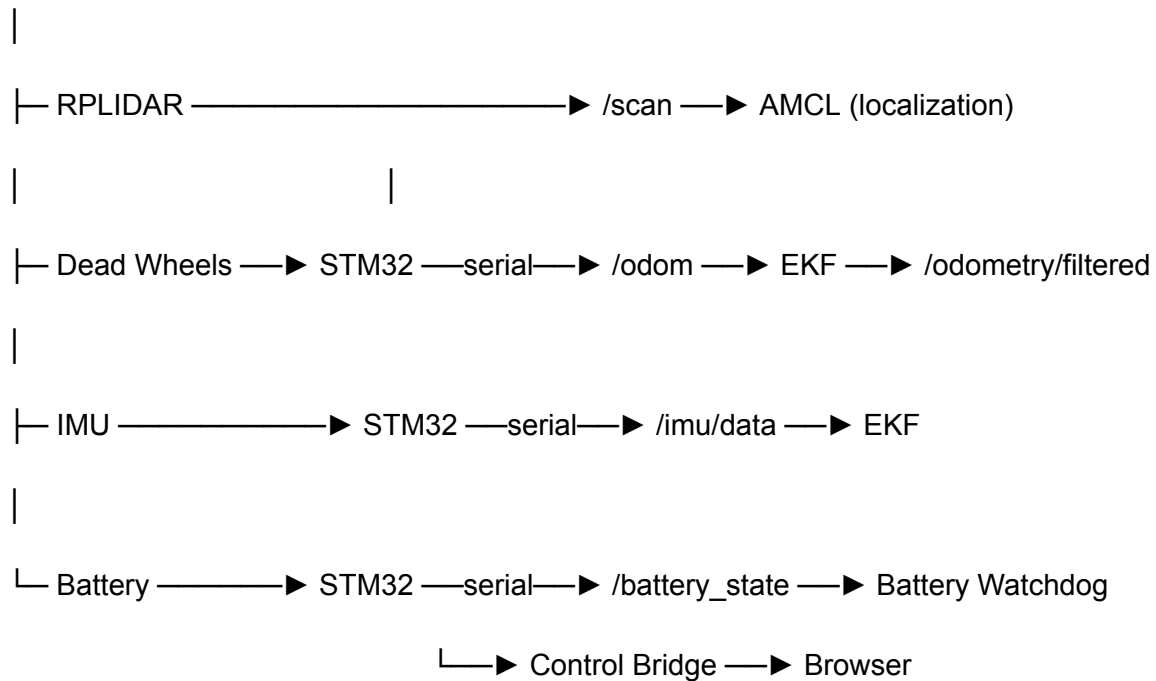
- Multi-channel audio for speech recognition

- Hardware direction-of-arrival (DOA) estimation accessible via USB vendor commands (used by `doa_server.py`)

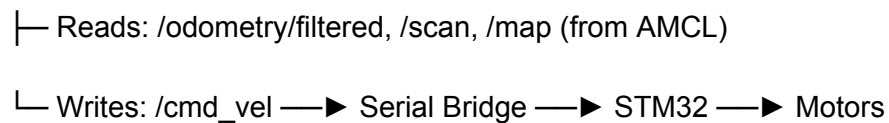
5. System Architecture

High-Level Data Flow

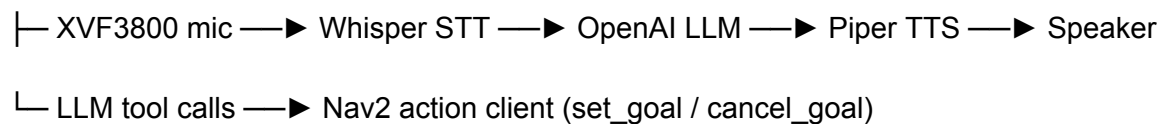
Physical World



Nav2 (path planner)



Voice Pipeline



Web Interfaces

└─ Browser Joystick ─► WebSocket (8765) ─► /cmd_vel

└─ Face (iPad port 8002) ◀─ React status UI ◀─ /sage/ui_state_json

└─ Face eyes ◀─ DOA WebSocket (8766) ◀─ XVF3800 azimuth

Process Map (tmux windows in `start_robot.sh`)

Window	Process	Purpose
0 — RS Publisher	<code>robot_state_publisher</code>	Broadcasts TF tree from URDF
1 — Valpo KB	<code>docker compose up</code>	Starts PostgreSQL + Django KB server
2 — Camera	<code>v4l2_camera_node</code>	USB camera → <code>/image_raw</code> (480×270)
3 — Video Server	<code>web_video_server</code>	Serves camera frames over HTTP
4 — Web Bridge	<code>control_bridge</code>	WebSocket ↔ <code>/cmd_vel</code> , status broadcast
5 — Teleop Web	<code>python -m http.server 8001</code>	Serves operator teleop UI
6 — Status Web	<code>npx vite</code> (port 8002)	Serves face/status React UI
7 — Serial Bridge	<code>serial_bridge_without_imu</code>	STM32 ↔ ROS 2 serial link
8 — Scan Publisher	<code>sllidar_a1_launch.py</code>	RPLIDAR → <code>/scan</code>
9 — Speech	<code>main.py</code>	Wake word, STT, LLM, TTS, Nav2 client
10 — AMCL	<code>localization_launch.py</code>	Map server + AMCL localization
11 — DOA	<code>doa_server.py</code>	XVF3800 DOA → WebSocket (8766)
12 — Battery Watchdog	<code>battery_watchdog.py</code>	Email alert on low battery

Window	Process	Purpose
13 — Nav2	<code>navigation_launch.py</code>	Path planner + behavior server

6. STM32 Firmware & Serial Protocol

The STM32G0B1 firmware lives on the `MCU` branch. It is a CubeIDE project targeting the STM32G0B1 MCU (ARM Cortex-M0+ @ 64 MHz).

What the STM32 Does

- Reads two **dead-wheel encoders** (passive odometry wheels) via hardware timers TIM1 and TIM2
- Runs a **PI control loop at 100 Hz** to drive both motors toward commanded velocities
- Reads the **BNO08x IMU** over I2C3 at 100 Hz
- Measures **battery voltage** via ADC on PA4
- Communicates bidirectionally with the Jetson over **USB CDC serial at 115200 baud**

Binary Serial Protocol

Every message has a 3-byte header followed by a payload:

Byte 0: SOF = 0x7E

Byte 1: TYPE (see below)

Byte 2: LEN (payload length in bytes)

Bytes 3+: PAYLOAD

All multi-byte values are **little-endian IEEE 754 floats**.

TYPE 0x01 — Command (Jetson → STM32)

Payload: 8 bytes

Offset	Type	Field	Units
0	float32	<code>v</code>	linear velocity (m/s)

Offset	Type	Field	Units
4	float32	w	angular velocity (rad/s)

- Sent by the serial bridge whenever `/cmd_vel` is published.
- The STM32 has a **550 ms watchdog**: if no command arrives within 550 ms, it ramps motors to zero.

TYPE 0x02 — Odometry (STM32 → Jetson)

Payload: 20 bytes, sent at **50 Hz**

Offset	Type	Field	Units
0	float32	x	position (m)
4	float32	y	position (m)
8	float32	theta	heading (rad, 0 to 2π)
12	float32	v	linear velocity (m/s)
16	float32	w	angular velocity (rad/s)

TYPE 0x03 — IMU (STM32 → Jetson)

Payload: 24 bytes, sent at **100 Hz**

Offset	Type	Field	Units
0	float32	gx	gyro X (rad/s)
4	float32	gy	gyro Y (rad/s)
8	float32	gz	gyro Z (rad/s)
12	float32	ax	accel X (m/s ²)
16	float32	ay	accel Y (m/s ²)
20	float32	az	accel Z (m/s ²)

TYPE 0x04 — Battery Voltage (STM32 → Jetson)

Payload: 4 bytes, sent at **50 Hz** (alongside odometry)

Offset	Type	Field	Units
0	float32	voltage	battery voltage (V)

The voltage is measured through a resistor divider ($100\Omega + 22\Omega$) on PA4. ADC full scale is 3.3V over 4095 counts; conversion factor is approximately 5.545 with a calibration multiplier of 0.9848.

STM32 RX State Machine

The STM32 listens for commands using a simple interrupt-driven state machine:

1. Hunt byte-by-byte for the command header byte **0x78** (120)
2. On match, collect the next 8 bytes within a 200 ms timeout
3. Parse the two floats; update **cmd_v** and **cmd_w**
4. Return to hunting
5. Any UART error (overflow, noise, framing) is cleared and reception restarts automatically

Kinematics & Odometry Constants

Constant	Value	Meaning
Lw	0.381 m	Wheel-to-wheel track width
DW_ENCODER_CPR	$600 \text{ CPR} \times 2 \text{ (2x mode)} = 1200$	Dead wheel encoder counts/rev
DW_WHEEL_DIAMETER	0.0814 m	Dead wheel effective diameter (calibrated)
DW_S_OFFSET_Y	0.1016 m	Straight dead wheel offset from robot center (lateral)
DW_H_OFFSET_X	0.400 m	Horizontal dead wheel offset from robot center (forward)
CTRL_HZ	100 Hz	Control loop frequency

The robot's heading is derived from the **horizontal dead wheel** ($v_H / \text{DW_H_OFFSET_X}$), and forward velocity from the **straight dead wheel** corrected by heading rate.

Motor Drive

Motors are driven by **ESC-style 50 Hz PWM** (TIM14 and TIM15). The center pulse width (1500 μ s duty count) means zero speed; 1700 is maximum forward; 1300 is maximum reverse. The serial bridge disables reverse driving (`max_linear_vel` is clamped on the ROS side so the robot never drives backward, which would look odd during tours).

7. ROS 2 Subsystems

Serial Bridge (`serial_bridge_without_imu.py`)

Located at

`ros2_ws/src/web_teleop_bridge/web_teleop_bridge/serial_bridge_without_imu.py`.

- Opens the STM32 serial port (auto-detects `/dev/ttyACM*` at 115200 baud)
- Subscribes to `/cmd_vel` → encodes TYPE_CMD frames → sends to STM32
- Receives TYPE_ODOM frames → publishes `nav_msgs/Odometry` to `/odom`
- Receives TYPE_BATTERY frames → publishes `sensor_msgs/BatteryState` to `/battery_state`
- Runs a dedicated RX thread for low-latency frame parsing
- Performs **IMU auto-calibration** on startup if the IMU bridge variant is used: the robot must be stationary for 5 seconds

The `serial_bridge.py` (with IMU) also publishes `/imu/data` from TYPE_IMU frames. The current default startup script uses `serial_bridge_without_imu.py` because EKF-based odometry fusion was found to perform well without the IMU in this environment. Use `start_robot_with_imu.sh` to enable IMU fusion.

Control Bridge (`control_bridge.py`)

Located at

`ros2_ws/src/web_teleop_bridge/web_teleop_bridge/control_bridge.py`.

An `asyncio`-based WebSocket server running on **port 8765** with two endpoints:

- `/ws/teleop` — Receives joystick/keyboard commands from the operator browser:

```
{"type": "control", "x": 0.3, "z": -0.5}
```

Publishes a `Twist` to `/cmd_vel`.

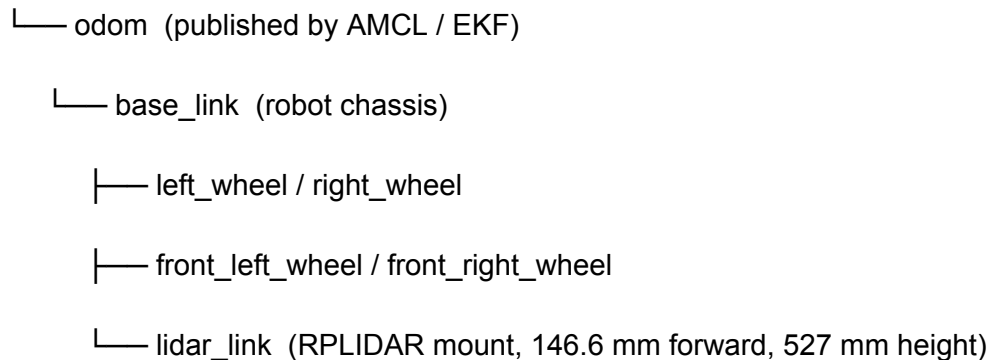
- `/ws/status` — Pushes robot state to connected browsers:
 - Battery voltage and percentage (from `/battery_state`)
 - UI state JSON (from `/sage/ui_state_json`)
 - Periodic ping/pong for connection health

Robot State Publisher

Reads `sagewithoutimu.urdf.xacro`, expands it, and broadcasts all fixed and joint transforms on the `/tf` and `/tf_static` topics. This is required for Nav2 and AMCL to know the sensor positions relative to `base_link`.

The TF tree looks like:

map



Extended Kalman Filter (EKF)

Configured in `config/ekf.yaml` using the `robot_localization` package.

- Fuses `/odom` (dead-wheel odometry) with `/imu/data` (angular rate) when using the IMU variant
- Without IMU, fuses odometry only for smoothing
- Output: `/odometry/filtered` at 50 Hz
- Publishes the `odom` → `base_link` transform

Navigation Stack

The navigation stack uses the pre-built map at `maps/new_save_map.yaml`.

AMCL (window 10):

- Launched via `nav2_bringup localization_launch.py`
- Uses `likelihood_field` laser model with up to 2000 particles
- Initial pose hardcoded in `config/nav2_params.yaml`: `x=25.45, y=2.21,  $\theta=0.0$` (approximately the SENIOR_DESIGN area, the usual development starting position)
- **After deployment at the docking station**, update the initial pose to match the docking station pose in the waypoints

Nav2 (window 13):

- Launched via `nav2_bringup navigation_launch.py`
- DWA local planner with costmaps (obstacle inflation radius tuned for SAGE's footprint)
- Behavior server with recovery plugins: spin, back-up, clear costmap
- Reverse driving disabled (robot always moves forward)

Nav2 and AMCL must be started **after** the `robot_state_publisher` and serial bridge are running, so that TF frames and odometry are available.

8. Navigation: Maps & Waypoints

Map Files

The active map of Gellersen Engineering's first floor is:

- `maps/new_save_map.pgm` — Grayscale occupancy grid image
- `maps/new_save_map.yaml` — Metadata (resolution, origin)

The map was built using **SLAM Toolbox** (online async mode) while manually teleoperating SAGE at very low speed (0.1 m/s) through the building. Use `slam_robot.sh` to rebuild the map if needed.

Waypoints

Waypoints are defined in two places that must be kept in sync:

- `maps/new_waypoints.yaml` — Poses used by Nav2 (position + quaternion orientation in the `map` frame)
- `speech/waypoints.py` — Same poses plus human-readable descriptions used by the LLM

Waypoint Key	Description
BIO_ENG_LAB	Bioengineering student project space
SENIOR_DESIGN	Senior design collaboration space (also the default starting pose)
GUELLY_DELLY	Engineering student café (breakfast/lunch)
MANUFACTURING_LAB	Manufacturing and fabrication lab
3D_PRINTING_LAB	3D printing and fabrication lab
CLEAN_ROOM	Controlled environment for sensitive fabrication
ECE_LAB_1	ECE lab with workbenches and tools
ECE_LAB_2	Second ECE lab
MECHATRONICS_LAB	Mechatronics projects and research
VALPO_ROBOTICS	Valpo Robotics student space
HESSE_CENTER	Tutoring and study center
BATHROOMS	Nearest restroom facilities
MATERIALS_TESTING_LAB	Materials science testing space
HEAT_POWER_LAB	Thermodynamics and heat transfer lab
TRANSPORTATION_LAB	Transportation engineering projects
DOCKING_STATION	Robot charging/docking location

Adding or Updating a Waypoint

1. Teleoperate SAGE to the desired location.
2. Open a terminal and execute `ros2 topic echo /amcl_pose` to read the current pose.
3. Add the pose to `maps/new_waypoints.yaml`.
4. Add a matching entry to `speech/waypoints.py` with a `description` field (this description is injected into the LLM system prompt).
5. Restart the speech process (window 9).

Alternatively, you can (this is actually how we did it):

Instructions on how to open Rviz2 in the debugging section #14 below. If you don't want the robot to be moving while you are doing this, you can kill all sage-related processes by executing `tmux kill-session -t sage`.

0. Kill sage-related processes by executing `tmux kill-session -t sage` (optional step)
1. Open Rviz2
2. Open another terminal and execute `ros2 topic echo /goal_pose`
3. On Rviz2, click on "set goal pose" and use the mouse to click on the pose (location and orientation) of your desired waypoint. The other terminal will log that pose. Then you can add it to `maps/new_waypoints.yaml` and match the entry in `speech/waypoints.py` with a description for this location.
4. If you did step 0, then move sage back to docking station, make sure it faces the wall/charging station, and reboot by turning on and off the switch at the bottom left of the robot.

Updating the Initial Pose for the Docking Station

When SAGE is deployed and always starts from the docking station, update `config/nav2_params.yaml` under the `amcl` section:

```
initial_pose_x: <docking_station_x>
```

```
initial_pose_y: <docking_station_y>
```

```
initial_pose_a: <docking_station_yaw>
```

The docking station pose in the current map is approximately `x=30.65`, `y=57.47`.

9. Voice Interaction System

All voice code lives in the `speech/` directory. The entry point is `speech/main.py`.

Pipeline Overview

XVF3800 USB mic array

|

DOA azimuth —————▶ doa_server.py (port 8766)

Audio stream ▼

Face UI (eyes follow speaker) ▼

openwakeword (ONNX)

Wakewords: "sage", "alexa", "hey jarvis"

(on detection) ▼

Silero VAD (voice activity detection)

(speech segment) ▼

faster-whisper (Whisper small.en, CUDA)

(transcript) ▼

OpenAI LLM (streaming, function calls)

┌──────────────────┐

Tool calls Text response

set_goal() |

cancel_goal() ▼

valpo_search() Piper TTS (local ONNX, en_US-amy-medium)

web_search() |

get_ip_address() ▼

| aplay (ALSA audio output)

|

▼

Nav2 action client (navigation.py)

Configuration ([speech/config.py](#))

All tunable parameters are centralized here:

Parameter	Default	Description
LLM_MODEL	gpt-5.4	OpenAI model for conversation
STT_MODEL	small.en	Whisper model (smaller = faster, English only)
WAKEWORD_SENSITIVITY	0.6	openwakeword detection threshold
VAD_SENSITIVITY	0.6	Silero VAD threshold
MAX_HISTORY_TURNS	12	LLM conversation history window
KB_SEARCH_URL	http://127.0.0.1:8004/api/kb/search	Knowledge base endpoint
PTT_PORT	8005	Push-to-talk web server port
TTS_MODEL	en_US-amy-medium.onnx	Piper TTS voice model

Wakeword

There are three active wakewords: Sage, Alexa, and Hey Jarvis. (located at `./speech/assets/models/wakeword`)

We trained the custom Sage wakeword using the simple Google Colab found at <https://github.com/dscripka/openWakeWord#training-new-models>. It worked well but struggled in ambient noise, hence we added "hey jarvis" and "alexa" that were made available by openwakeword and were trained on more data.

To be fair, the training data for the sage wakeword was entirely synthetic. We could have added samples with recordings of the ambient noise in Gellersen.

LLM Persona & System Prompt (`speech/system_prompt.py`)

SAGE's persona is **"SAGE Jarvis"** — a friendly, witty, and concise tour guide. The system prompt is rebuilt before each conversation turn and includes:

- Today's date
- All waypoint names and descriptions (injected dynamically from `waypoints.py`)
- Current navigation status (navigating / idle, distance remaining, target name)

The LLM is instructed to:

- Set only one goal at a time, never chain destinations without arrival confirmation
- Use plain English in responses (no symbols or links — TTS cannot speak them)
- Use `valpo_search` first for university-specific questions
- Use `web_search` for general knowledge or current events
- Respond to arrival event prompts with a welcome message

LLM Tools

Defined in `speech/tools.py`:

Tool	Description
<code>set_goal(location)</code>	Navigate to a named waypoint (non-blocking)
<code>cancel_goal()</code>	Stop current navigation
<code>valpo_search(query, top_k)</code>	Semantic search in the local knowledge base
<code>web_search(query, max_results)</code>	Tavily internet search

Tool	Description
<code>get_ip_address()</code>	Return Jetson network interfaces

Web Push-to-Talk (`speech/web_ptt.py`, port 8005)

For users who cannot or prefer not to speak directly to SAGE:

1. Browser records audio using MediaRecorder (WebM format)
2. Audio POSTed to `/speech/audio`
3. Server converts WebM → 16 kHz WAV using `ffmpeg`
4. `faster-whisper` transcribes locally (singleton model, lazy-loaded)
5. Transcript fed into the same LLM pipeline
6. Response returned as JSON with transcript and TTS audio

Direction-of-Arrival Server (`speech/doa_server.py`, port 8766)

Reads the **XVF3800 mic array's AEC azimuth** via USB vendor control transfer and broadcasts the angle over WebSocket to the face UI, causing SAGE's animated eyes to follow the direction of speech.

- Angle convention: 0° = behind robot, 90° = left, 180° = in front, 270° = right
- Broadcast rate: ~12.5 Hz (80 ms interval)
- Falls back to a stationary angle in simulation mode if the mic is not found

Knowledge Base (`knowledge_base/`)

The knowledge base runs as a Docker Compose stack (PostgreSQL + Django web service).

- **Django Environment:** `knowledge_base/.env.template` contains the template for the `.env` file in the same directory that defines environment variables for the knowledge base.
- **Content:** Tour talking points document (~8 pages) provided by Dean Doug Tougaw of the College of Engineering. The KB is sparse and would benefit from more ingested content (Valpo links, program descriptions, faculty, etc.).
- **API port:** 8004
- **Search endpoint:** `POST /api/kb/search` with `{"query": "...", "top_k": 5}`
- **Email endpoint:** `POST /api/kb/send-emails` (used by the battery watchdog)
- **Embeddings:** OpenAI API (requires `OPENAI_API_KEY` set in the Django environment)
- **Admin panel:** `http://<sage-ip>:8004/admin/`

To add documents to the KB, go to the admin panel `http://<sage-ip>:8004/admin` and upload the new documents. They will be automatically ingested. You will be prompted for a username and password before login in, you can ask the administration to provide it to you.

The KB data is **not committed to the repository**. A new deployment must re-ingest documents.

10. Web Interfaces & Network Services

Port Reference

Port	Service	Used By
80	Landing page: lists services with links to them	
8001	Teleop interface (HTTP)	Operator browser
8002	Face / status interface (Vite/React)	iPad in kiosk mode
8004	Knowledge base API (Django)	Speech system, battery watchdog
8005	Push-to-talk web endpoint (FastAPI)	Visitor browser
8765	Control bridge WebSocket	Teleop interface, status interface
8766	DOA WebSocket	Face / status interface
8080*	web_video_server	Teleop interface (video stream)

*web_video_server uses its default port. The teleop interface fetches the stream from http://<sage-ip>:8080/stream?topic=/image_raw.

Teleop Interface (port 8001)

A single-page HTML/JS application served by Python's built-in HTTP server. Features:

- Live camera feed (480×270)
- Virtual steering wheel + throttle pedals
- Keyboard arrow key support

- Battery voltage and percentage display
- UI state indicator (what SAGE is doing)
- Connects to control bridge WebSocket at port 8765

Face / Status Interface (port 8002)

A React 19 + TypeScript + Vite application. This is what the **iPad displays in kiosk mode**, mounted on the front of the robot. It shows SAGE's animated face and receives:

- Robot UI state from control bridge WebSocket (port 8765)
- Speaker direction from DOA WebSocket (port 8766) — moves the pupils to follow the speaker

Push-to-Talk Page (port 8005)

Accessible from any browser on the same network. Visitors can hold a button to speak or type a message, and SAGE responds via its speaker. The FastAPI server at this address handles the STT → LLM → TTS pipeline.

11. Startup Procedures

Simple Startup

Sage by runs the `./start_robot.sh` script on boot, so you shouldn't have to worry about starting sage besides flipping the switch.

Make sure SAGE is at her docking station facing the wall/charger. Power down SAGE using the switch located at the bottom left of the robot. Then turn it ON. After a few minutes (~5), she should be ready to go.

Prerequisites

Before running any startup script, verify:

1. SAGE is powered on and the Jetson has booted
2. STM32 is connected via USB (should appear as `/dev/ttyACM0` or `/dev/ttyACM1`)
3. RPLIDAR is connected via USB
4. XVF3800 mic array is connected via USB
5. USB camera is connected
6. SAGE is on the WiFi network
7. Docker is running (for the knowledge base)
8. API key files are in place (see Section 12)

Full System Startup (Normal Operation)

You can always verify if sage-related processes have already been started by running `tmux ls`. If something is already running, you can kill it using `tmux kill-session -t sage`.

After making sure there are sage-related processes running, you can manually start it using the bash script as follows:

```
cd ~/Desktop/SAGE_ROBOT
```

```
./start_robot.sh
```

This creates a `tmux` session named `sage` with 14 windows (0–13). To attach:

```
tmux attach -t sage
```

Navigate between windows: `Ctrl+B` then the window number (0–9) or `Ctrl+B n/p` for next/previous.

To stop everything:

```
tmux kill-session -t sage
```

SLAM Mode (Building a New Map)

Use this when the map needs to be rebuilt (e.g., after significant furniture changes):

1. Kill the running sage-related processes using `tmux kill-session -t sage`
2. Place the robot at good starting point (that is going to be the origin of the map). We used the bio engineering lab for our origin.
3. Execute `slam_robot.sh` to start up the SLAM processes.
4. Wait for a few minutes (~5), then drive the robot very slowly around the building using the teleop interface (set speed to 10%) which corresponds to 0.1m/s.
5. You can view the map being built live by opening Rviz2.

Note Rviz2 on SAGE already has saved settings, so you shouldn't have to set it up. But if on a new machine, then you will have to set it up yourself. (AI can help you with this and there are YOUTUBE videos that are in the resources section below)

This starts SLAM Toolbox in online async mode instead of AMCL. Teleoperate SAGE through the entire area to be mapped. Save the map with:

```
ros2 service call /slam_toolbox/save_map slam_toolbox/srv/SaveMap "{name: {data:
'/home/agi/Desktop/SAGE_ROBOT/maps/new_save_map'}}"
```

Alternatively you can save the map directly from Rviz2: In RViz2, go to Panels -> Add New Panel. Select SlamToolboxPlugin. Use the Save Map button in the panel to save your current session

After saving, re-establish waypoints by teleoperating to each location and reading the pose.

Important: go to the directory where you saved the map (/home/agi/Desktop/SAGE_ROBOT/maps/), open the yaml file and reduce free threshold to 0.1.

IMU-Enabled Startup

Same as the steps above but in step 3 use the following command instead:

```
./start_robot_with_imu.sh
```

Requires `serial_bridge.py` (instead of `serial_bridge_without_imu.py`) and the IMU to be properly calibrated. The robot must remain **stationary for 5 seconds** after the serial bridge starts for IMU bias calibration.

Restarting a Single Window

If one process crashes, you can restart just that window without rebooting everything. Attach to the tmux session, switch to the crashed window, and re-run its command. Each window runs `exec bash` at the end so you get a shell after a crash.

Example — restart the speech system:

```
tmux attach -t sage
```

```
# Switch to window 9 (Speech): Ctrl+B then 9
```

```
source /opt/ros/humble/setup.bash
```

```
source ~/Desktop/SAGE_ROBOT/ros2_ws/install/local_setup.bash
```

```
source ~/.venv/bin/activate
```

```
cd ~/Desktop/SAGE_ROBOT/speech
```

```
python main.py
```

12. API Keys & Secrets

Keys are stored in `speech/api_keys/api_keys.json`. This file is **not committed to the repository** (listed in `.gitignore`).

Required keys:

```
{  
  
  "OPENAI_API_KEY": "sk-...",  
  
  "TAVILY_API_KEY": "tvly-..."  
}
```

How to obtain:

- **OpenAI API key:** Create an account at platform.openai.com → API Keys → Create new secret key
- **Tavily API key:** Create an account at tavily.com → API → generate key (used for `web_search` tool)

The knowledge base Django service also needs the OpenAI key set as an environment variable in `knowledge_base/docker-compose.yml` (or a `.env` file in the `knowledge_base/` directory) for embedding ingested documents.

13. Libraries Installation & Modifications

`python_3_10_12_requirements.txt` contains the libraries and their versions.

So run `pip3 install -r python_3_10_12_requirements.txt`

Now, that's not enough because the torch libraries installed won't be installed with cuda wheels, so the speech processing will be slow.

installing torch with cuda wheels

The speech processing is relatively quick because we are using the GPU cores on the jetson nano. If trying to reproduce this project, make sure you install torch with cuda wheels on your device.

```
pip install torch torchvision torchaudio --index-url https://pypi.jetson-ai-lab.dev/jp6/cu126/+simple
```

We used so Claude web search to to help us find the right link to install torch with cuda on the Jetson nano orin. We also had to downgrade numpy

```
pip install "numpy<2"
```

And use this specific version of torchaudio

```
pip uninstall torchaudio -y
```

```
pip install torchaudio==2.8.0 --index-url https://pypi.jetson-ai-lab.io/jp6/cu126
```

realtimestt — audio_recorder.py Patch

The upstream `realtimestt` library has bugs in its `audio_recorder.py` that caused issues on the Jetson. A patched version is saved at:

```
speech/replace_in_stt_library/audio_recorder.py
```

After every `pip install` or `pip upgrade of realtimestt`, replace the installed file:

```
# Find the installed location
```

```
SITE=$(python -c "import site; print(site.getsitepackages()[0])")
```

```
cp ~/Desktop/SAGE_ROBOT/speech/replace_in_stt_library/audio_recorder.py \
```

```
    $SITE/RealtimeSTT/audio_recorder.py
```

Or with the venv active:

```
source ~/.venv/bin/activate
```

```
SITE=$(python -c "import site; print(site.getsitepackages()[0])")
```

```
cp ~/Desktop/SAGE_ROBOT/speech/replace_in_stt_library/audio_recorder.py \
```

`$SITE/RealtimeSTT/audio_recorder.py`

Failing to apply this patch will likely cause silent failures or crashes in the speech pipeline.

14. Known Issues & Debugging Guide

How to Open Rviz2 and the remote desktop GUI in general

1. Make sure the robot is ON.
2. Open Remote Desktop on Windows or Windows App on Mac OS
3. Connect to the robot using its IP
4. You will be prompted to enter the username and password. If you do not have this information, ask the administration.
5. Now you have access to the remote GUI. To open Rviz2, open a terminal, and execute `rviz2`.

To minimize memory usage, we are using a substitute for the Gnome Desktop (the default GUI of the jetson) that only comes ON when we remote desktop into the Robot.

Diagnosing a Problem

When something is wrong, the best first step is:

1. Attach to the tmux session: `tmux attach -t sage`
2. Cycle through windows (Ctrl+B, 0–13) looking for red error output
3. Note which window is failing and read the traceback

Resolving a Problem

The best advice here is to reboot the system! Simple but effective.

1. Teleop sage to docking station, ensure it faces the wall/charging station.
2. Turn OFF the entire system by flipping the switch at the bottom left of the robot.
3. Wait for ~5mins and turn it back ON.
4. Wait for ~5mins and the robot should be good to go.

Issue: Robot Stops Navigating When People Get Too Close or Walls

Symptom: SAGE aborts navigation mid-route when someone walks very close to it. The LIDAR sees them as a sudden obstacle inside the inflation radius. This can also happen if SAGE got too close to a wall while navigating or avoiding an obstacle.

Recovery: Re-prompt SAGE verbally with the same destination. It will replan from the current position. If too close to a wall, prompt for a destination that is in a direction away from the wall or just push it away from the wall by about 0.5m and prompt for destination again.

Root cause: Nav2's costmap inflates obstacles; a person standing very close is treated as a fatal obstacle. This is intentional safety behavior. The `inflation_radius` in `config/nav2_params.yaml` can be reduced if this happens too frequently, but doing so risks SAGE getting closer to walls.

Issue: Robot Steers Toward Walls / Wobbly Movement

Symptom: SAGE drifts sideways, or AMCL localization jumps erratically.

Root cause (historical, now fixed): One or both dead-wheel encoder wheels were slightly tilted (not perfectly perpendicular to the direction of travel). This caused odometry to drift, which confused AMCL and led to off-center path planning. The wheels were re-secured with thread-locker (Loctite).

If this recurs: Inspect the dead wheels for tilt. They must be **perfectly perpendicular** — the straight wheel parallel to the robot's forward axis, and the horizontal wheel parallel to the lateral axis. Re-secure loose wheels with Loctite.

Also, you may want to double-check `config/nav2_params.yaml` and make sure that the regulated pure pursuit controller has a `lookahead_distance` set to at least 1.2. You can use AI to explain you explain those parameters and tweak them.

Diagnostic: Echo odometry while the robot is stationary: `ros2 topic echo /odom`. The `x`, `y`, `theta` values should not drift when the robot is not moving. The `y` and `theta` values should not be changing if the robot is moving straight forward. You can also use a tape to measure the actual distance moved in each direction (`x` is forward, and `y` is sideways) and check if they agree with the `/odom`.

Issue: Path Planner Generates a Path Behind the Robot

Symptom: SAGE gets near an obstacle and then stops, apparently unwilling to move even after the obstacle is clear. Setting a new goal fixes it.

Root cause (historical, now fixed): The behavior server recovery plugins were not configured, so Nav2 could not recover from near-collision states on its own. After adding the spin, back-up, and clear-costmap recovery plugins to `nav2_params.yaml`, this behavior was resolved.

If it recurs: Check the Nav2 window (13) for error messages. Try cancelling the goal via voice ("cancel the goal") and setting a new one. If Nav2 is stuck in a bad state, restart window 13 or just drive the robot to docking station, then reboot using the switch (it has to be booted from docking station because that's where the initial pose is set to be).

Issue: Robot Cannot Navigate in a Crowded Hallway

Symptom: Nav2 cannot find a valid path; reports "no path found" or goal is aborted immediately.

Root cause: SAGE's footprint + inflation radius requires a minimum corridor width. With many people in the hallway, the costmap sees no passable space. SAGE is also tall and cannot detect obstacles shorter than the LIDAR mounting height (~527 mm), so it may unknowingly approach low obstacles or people's feet.

Mitigation: Ask people to move aside. The inflation radius can be tuned in `nav2_params.yaml`, but note the trade-off with wall proximity.

Issue: Speech System Not Responding

Symptom: SAGE does not react to the wake word or voice commands.

Check window 9 (Speech) for:

- `CUDA out of memory` — Whisper model loaded on GPU but memory is full; try rebooting
- `No audio device found` — PulseAudio is not running; the startup script exports `PULSE_SERVER` but it may need to be started manually: `pulseaudio --start`
- `OpenAI API error` — Check the API key in `api_keys.json` and account quota
- `Connection refused` on KB search — Knowledge base Docker containers are not running; check window 1

Issue: Knowledge Base Not Responding

Symptom: LLM tool calls to `valpo_search` return errors; battery watchdog email fails.

Check window 1 (Valpo KB):

```
docker compose ps # in knowledge_base/
```

docker compose logs

If containers are not running: `docker compose up -d`

Issue: Teleop Interface Shows No Video

Symptom: The browser at port 8001 shows the UI but the camera feed is black or absent.

Check:

- Window 2 (Camera): Is `v4l2_camera_node` running without errors?
- Window 3 (Video Server): Is `web_video_server` running?
- Is the USB camera detected? `ls /dev/video*`

Issue: iPad Face Not Showing

Symptom: iPad shows a blank page or connection error.

Check:

- Window 6 (Status Web): Is Vite running on port 8002 without errors?
- Is the iPad on the same WiFi network as SAGE?
- Navigate the iPad browser to `http://<sage-ip>:8002`

Issue: STM32 Not Communicating

Symptom: Serial bridge crashes or `/odom` is not published; robot wheels do not respond to commands.

Check window 7 (Serial Bridge):

- `ls /dev/ttyACM*` — Is the STM32 enumerated?
- Unplug and replug the USB-C cable between the Jetson and the STM32
- Check `config.py` or the serial bridge code for the correct serial port path
- The STM32 LED (PA5, green) should blink; if it is off, the MCU may not be powered or running

Issue: AMCL Localization Jumps / Robot Is Lost

Symptom: In RViz2, the particle cloud is spread out or in the wrong place; the robot drives as if it does not know where it is.

Recovery:

1. Teleoperate SAGE to a recognizable location
2. Use the "2D Pose Estimate" tool in RViz2 to set the approximate pose, or update `initial_pose_x/y/a` in `nav2_params.yaml` and restart window 10 (AMCL)
3. Spin the robot slowly in place to help AMCL converge

Root cause: AMCL can lose localization if odometry drifts significantly (see dead-wheel issue above) or if the environment has changed substantially since the map was built (furniture moved, new obstacles). This can also happen if the scan window (height and width of `local_costmap` in `nav2_params.yaml` is too small). It used to get lost around Guelly Delly when the window size was set to 3. When we increased to 5, the issue was resolved.

Alternatively, just reboot the robot from the docking station (it has to be booted from there because that's where the initial pose is set to be).

15. Porting to Another Robot

SAGE's software stack can be adapted to any differential-drive robot. The boundaries of what needs to change are clear:

Physical / Mechanical Layer (must change)

Update the URDF in `description/` to match the new robot's geometry: link dimensions, sensor positions, wheel track width, and wheel radius. Keep the same link names (`base_link`, `left_wheel`, `right_wheel`, `lidar_link`) so Nav2 and AMCL work without further changes.

Serial Communication Layer (may change)

If the new robot uses a different MCU or firmware, update the serial protocol in `serial_bridge_without_imu.py`. The frame format (SOF + TYPE + LEN + PAYLOAD) is simple and can be adapted. The key requirement is that the bridge publishes:

- `nav_msgs/Odometry` to `/odom` at ≥ 20 Hz
- `sensor_msgs/BatteryState` to `/battery_state` (optional but recommended)
- Subscribes to `geometry_msgs/Twist` on `/cmd_vel`

Navigation Layer (minimal changes)

Rebuild the map with SLAM Toolbox, then update the waypoint poses in `maps/new_waypoints.yaml` and `speech/waypoints.py`. Update `config/nav2_params.yaml` — particularly `robot_radius`, `inflation_radius`, and initial pose — to match the new robot's footprint.

Voice & LLM Layer (super minimal changes required)

The speech system, knowledge base, and web interfaces are hardware-agnostic. Only the api keys, waypoint descriptions, and system prompt persona need updating for a different environment.

16. Useful Resources

The following YouTube playlist was helpful:

<https://youtube.com/playlist?list=PLunhqkrRNRhYAffV8JDIFoatQXuU-NnxT&si=fObfoAWOqSp-yoU>. It goes over some foundational concepts and terms. We skimmed over some videos like the simulation and teleop because we didn't want to waste time with simulations and our teleop approach was completely different, but overall was a useful playlist.

17. Acknowledgments

- Valparaiso University College of Engineering — facilities, support, and tour content
- Dean Doug Tougaw, our customer, — provided funding and the Tour Talking Points document for the knowledge base
- ROS 2, Nav2, SLAM Toolbox, and robot_localization open-source communities
- OpenAI, Piper TTS, faster-whisper, openwakeword, and realltimeTT projects
- All faculty (especially Dr Georges El-Howayek our supervisor), students, and collaborators involved in testing and integration
- Fayol Ateufack (CE) led the team, worked on the development and integration of navigation, speech, interfaces, and knowledge base + search queries software stack.
- Aidan Matson (ME) was the CTO, worked on designing + 3D printing the frame, coding the MCU, mounting the wheels, designing PCB and overseeing battery safety + charging efforts.
- Ranger Scott (EE) assembled the battery cells, designed the charging circuit, integrated battery chip, built the charger, printed the contacts.
- Samuel Starkenburg (ME) designed and printed the lid with integrated microphone, lidar, and camera as one seamless unit. Oversaw the design, printing, and setup of charging station's mechanical components.
- Tobias Demonte (ME) worked on the design, printing, and testing of previous iterations of wheels. Designed and printed mounts for the speaker and battery. Led documentation efforts.

- Zach Nieslen (CE) researched on speech components like the microphone, integrated the IMU sensor, contributed to design choices, and managed internal and external communication (emails, posters, presentations).

Developed as part of Valparaiso University's Senior Design Program. [Video Demo](#)